

Gluon W: A Cryptocurrency Stabilization Protocol

Zahnentferner ✉

The Stable Order

Luca D'Angelo ✉

zenGate Global

The Stable Order

Mohammad Shaheer ✉

Carnegie Mellon University in Qatar

The Stable Order

Giselle Reis ✉

Carnegie Mellon University in Qatar

Abstract

This paper introduces Gluon W, a novel stablecoin protocol inspired by nuclear physics and named after the particle responsible for the stability of matter in the universe. The key idea in Gluon W is to split (as in nuclear fission) an existing volatile asset into its stable and unstable components. These components can be merged back (as in nuclear fusion) into the original asset or transmuted into each other (as in nuclear beta decays). Various stability theorems are proven and their proofs are formally verified using the interactive proof assistant *Rocq*.

2012 ACM Subject Classification Applied computing → Economics

Keywords and phrases Blockchain, Cryptocurrency, Stablecoin

Acknowledgements We are grateful to the Sui Foundation and Cardano's Catalyst for grants that partially supported this research. We would like to thank: Kii, Luca D'Angelo, Sangeet, Hammerman and nkz for the implementation on Ergo; Krasavice Blasen for the Python Simulation and the audit of the Ergo implementation; Michael for testing the Ergo implementation. The main responsibilities of the authors were: Zahnentferner – invention and description of Gluon and supervision of the work on proving the stability theorems; Luca D'Angelo – proofs of the stability theorems; Mohammad Shaheer – formal verification of the proofs in Rocq; Giselle Reis – supervision and mentoring of the formal verification.

1 Introduction

For cryptocurrencies to actually be proper currencies, they need to perform well the functions of money. They must be capable of serving as a medium of exchange, unit of account and store of value. All these functions require stability of their value relative to the goods and services for which one might use them. However, traditional cryptocurrencies, such as BTC, have been plagued by volatility, since their monetary policies are rigid and unable to cope with fluctuations in demand. To address this volatility problem, the concept of *stablecoin* has arisen. Stablecoins aim to have a stable value in relation to another asset, typically (but not necessarily) a fiat currency, such as the USD.

The most popular stablecoins by market cap are fiat-backed: they are backed by roughly the same asset as the asset that their price is pegged to. For example, USD stablecoins (such as USDT and USDC) are backed by USD in a bank account or low-risk USD-denominated bonds. Some, but not necessarily all, holders of such stablecoins have a right to redeem their stablecoins for actual USD in their bank accounts. Such fiat-backed stablecoins, however, need to be centrally managed by entities whose reserves are opaque and whose business models rely on the interest rate of the bonds in which they invest the reserves and thus

depend on the very inflation of the supply of fiat currencies that was a major motivating factor for the invention and pursuit of cryptocurrencies. Furthermore, because their profit increases almost linearly with the amount of reserves controlled by these entities but their costs do not, there is a tendency for the entities issuing fiat-backed stablecoins with large market caps to outcompete entities issuing fiat-backed stablecoins with small market caps. This leads to further centralization of the control of vast amounts of fiat currency reserves. This, and the financial risks that it entails, should be concerning not only to those who are philosophically aligned with the decentralization principles of cryptocurrencies, but also for regulators who care about financial and economic stability.

In the past decade, although they are currently not as popular as centralized fiat-backed stablecoins, many decentralized stablecoin protocols have been proposed, implemented, deployed and used. They can be classified in a few different categories [9], such as: seigniorage-based [5] (e.g. Terra), crypto-collateralized (e.g. DAI [1]) and crypto-backed (e.g. Djed [10, 11]). While seigniorage-based stablecoin protocols have proven to be fatally flawed with the collapse of Terra, crypto-collateralized and crypto-backed stablecoins have proven to be able to maintain their stability despite severe shocks in the price of their collateral and reserve assets.

Crypto-backed stablecoins, for instance, function similarly to fiat-backed stablecoins in the sense that the stablecoin holder also has a right of redemption. But the stablecoins can be redeemed not for the fiat currency for which the stablecoin is pegged to, but for an amount of a cryptocurrency held by a smart contract as a reserve asset. Since cryptocurrencies are volatile, the reserve ratio is kept at levels significantly higher than 100%, so that full-value redemption remains possible even after significant drops in the price of the reserve asset.

The Gluon W stablecoin protocol belongs to an entirely new category of stablecoin protocols. Unlike crypto-backed stablecoins, which issue a new currency that is backed by a cryptocurrency, Gluon W allows users to take the existing cryptocurrency and split it into two sub-assets: one of them is stable w.r.t. a target peg asset and the other one is even more volatile than the base cryptocurrency.

This splitting mechanism is inspired by the nuclear physics reaction of *fission*. Taking this physics analogy further, Gluon W allows users to merge the two sub-assets back together to obtain the original cryptocurrency, similarly to nuclear *fusion*; and one sub-asset may be transmuted into the other, similarly to nuclear *beta decays* of protons to neutrons and neutrons to protons.

The Gluon W protocol can also be seen as a dual of liquidity pools. Whereas in a liquidity pool, one deposits two assets into a pool and obtains an LP token that represents a share of the pool, in Gluon W one deposits the base cryptocurrency and receives two (sub-)assets that represent a share of the pool. Redeeming the LP token gives the user his/her an amount of the two assets back. Likewise, conversely, by fusing the two (sub-)assets the user gets an amount of base cryptocurrency back.

As in the case of liquidity pools, which have mechanisms to discourage large swaps between the two assets, Gluon W has mechanisms to discourage large beta decays of one sub-asset into the other. Unlike liquidity pools, though, the amounts of base cryptocurrency represented by each sub-asset depend also on an external oracle.

This paper has the following structure. In Section 2, the Gluon W protocol is precisely described. In Section 3, various theorems about Gluon W's stability are mathematically proven. In Section 4, the formal verification of the proofs of these theorems using the interactive proof assistant Rocq [4] is discussed. In Section 5, a comparison with the Djed Stablecoin Protocol is made. In Section 6, current implementations of the protocol are

described.

2 Protocol Description

2.1 Definitions

The essence of Gluon W is that, analogously to how an atom's nucleus is composed of protons and neutrons (known collectively as nucleons), a token, denoted by $\mathcal{G}$, is composed of two sub-assets (named collectively as *tokens*):

- : *stable tokens* or *neutrons*, whose price is kept stable relative to a target price (e.g. a fiat currency or commodity).
- : *volatile tokens* or *protons*, whose price is more volatile than $\mathcal{G}$'s.

The protocol defines the rules of an autonomous *reactor* capable of four reactions:

1. **Fission** The user sends $\mathcal{G}$ to the reactor and receives neutrons and protons.
2. **Fusion** The user sends neutrons and protons to the reactor and receives $\mathcal{G}$.
3. **Beta decay** β^+ ($\bullet \Rightarrow \circ$): The user sends protons to the reactor and receives neutrons.
4. **Beta decay** β^- ($\circ \Rightarrow \bullet$): The user sends neutrons to the reactor and receives protons.

The reactor's state is a triple $(R, S_\circ, S_\bullet)$ where:

- R is the number of $\mathcal{G}$ that is currently fissioned, also called the *reserve*,
- $S_\circ$ is the number of neutrons in circulation,
- $S_\bullet$ is the number of protons in circulation.

In order to determine the quantities involved in each reaction, the protocol is parametrized by the following values, which are set at the time of deployment and never changed.

q^* : Upper bound for fusion ratio.

Constraint: $0 < q^* < 1$

$\Phi_\downarrow$: Intuitively denotes the intrinsic cost for a *fission*.

Constraint: $0 < \Phi_\downarrow < 1$

$\Phi_\uparrow$: Intuitively denotes the intrinsic cost for a *fusion*.

Constraint: $0 < \Phi_\uparrow < 1$

Φ_β^x ($x \in \{0, 1\}$): y-intercept and slope of the linear function used to determine the cost of beta decays.

Constraints: $0 < \Phi_\beta^x \leq 1$ and $0 < \Phi_\beta^0 + \Phi_\beta^1 \leq 1$.

T : A fixed length of time. The cost of a beta decay takes into account the volume of beta decays within a time window of this length.

Besides the constant parameters, the protocol also depends on a variable *target price* for neutrons, denoted $P_\circ^*$ and assumed to be given by an independent external oracle.

The *fusion ratio* determines what fraction of the reserve is backing neutrons. The *neutron price* is that amount of reserve per neutron, and the *proton price* is the amount of reserve per proton. These three important notions are defined by the following equations.

■ Fusion ratio

$$q(R, S_\circ) = \min\left(q^*, \frac{S_\circ P_\circ^*}{R}\right)$$

■ Neutron price

$$P_{\circ}(R, S_{\circ}) = \frac{q(R, S_{\circ})R}{S_{\circ}}$$

■ Proton price

$$P_{\bullet}(R, S_{\circ}, S_{\bullet}) = \frac{(1 - q(R, S_{\circ}))R}{S_{\bullet}}$$

2.2 Beta decay fees

At a given time τ , the beta decay fees for β^+ ($\bullet \Rightarrow \circ$) and β^- ($\circ \Rightarrow \bullet$) are, respectively:

$$\begin{aligned}\varphi_{\beta^+}(\tau) &= \Phi_{\beta}^0 + \Phi_{\beta}^1 \frac{\max\{\Delta(\tau), 0\}}{R} \\ \varphi_{\beta^-}(\tau) &= \Phi_{\beta}^0 + \Phi_{\beta}^1 \frac{\max\{-\Delta(\tau), 0\}}{R}\end{aligned}$$

$\Delta(\tau)$ is defined as:

$$\Delta(\tau) = \sum_{r \in \chi_+(\tau)} \nu(r) - \sum_{r \in \chi_-(\tau)} \nu(r)$$

Where $\chi_+(\tau)$ and $\chi_-(\tau)$ are the sets of beta decays β^+ and β^- , respectively, between times $(\tau - T)$ and τ , and $\nu(r)$ is the volume in $\mathfrak{Q}$ of a beta decay reaction r . The volume of a β^+ reaction r^+ is defined as $\nu(r^+) = P_{\bullet} n_{\bullet}$, where $P_{\bullet}$ is the current proton price and $n_{\bullet}$ is the amount of protons being converted to neutrons. Analogously, the volume of a β^- reaction r^- is $\nu(r^-) = P_{\circ} n_{\circ}$.

In other words, $\Delta(\tau)$ is the balance of beta decay reactions in the time window up to τ .

Note in particular that, if the balance is positive, that is, there was a bigger volume of β^+ decays than β^- decays, then the fee φ_{β^-} is Φ_{β}^0 , while the fee φ_{β^+} for the dual reaction is higher. The use of *max* ensures that the fee is never negative.

2.3 Reactions

The inputs (tokens or tokenons that the users provides) and outputs (tokens or tokenons that the user receives) of Gluon W's reactions are defined in Tables 1, 2 and 3. These three tables are mathematically equivalent. One can be transformed into the others by expanding definitions and performing algebraic simplifications. The three tables are shown here for convenience, since some of these equations are easier to implement than others, depending on the implementation approach taken, and also for the sake of providing additional insight into the protocol's behavior. For instance, Tables 2 and 3 make it clear that fission and fusion do not depend on the target price.

2.4 Example Calculation

For example, suppose the current reactor state is: $R = 1$, $S_{\circ} = 2$, and $S_{\bullet} = 1$. And suppose that the fission fee is 0.1. If a user sends 10 $\mathfrak{Q}$ to the reactor, the user receives 18 neutrons and 9 protons, and the new reactor state will be $R = 11$, $S_{\circ} = 20$, and $S_{\bullet} = 10$.

■ **Table 1** Gluon W's Reactions (v1)

Reaction	Inputs	Outputs
Fission	$m \textcircled{2}$	$\frac{m(1-q(R,S_o))(1-\Phi_{\downarrow})}{P_{\bullet}(R,S_o,S_{\bullet})} \bullet$ $\frac{m q(R,S_o)(1-\Phi_{\downarrow})}{P_o(R,S_o)} \circ$
Fusion	$\frac{m(1-q(R,S_o))}{P_{\bullet}(R,S_o,S_{\bullet})} \bullet$ $\frac{m q(R,S_o)}{P_o(R,S_o)} \circ$	$m(1-\Phi_{\uparrow}) \textcircled{2}$
β^+	$\frac{m}{P_{\bullet}(R,S_o,S_{\bullet})} \bullet$	$\frac{m(1-\varphi_{\beta^+}(\tau))}{P_o(R,S_o)} \circ$
β^-	$\frac{m}{P_o(R,S_o)} \circ$	$\frac{m(1-\varphi_{\beta^-}(\tau))}{P_{\bullet}(R,S_o,S_{\bullet})} \bullet$

■ **Table 2** Gluon W's Reactions (v2)

Reaction	Inputs	Outputs
Fission	$m \textcircled{2}$	$m(1-\Phi_{\downarrow}) \frac{S_{\bullet}}{R} \bullet$ $m(1-\Phi_{\downarrow}) \frac{S_o}{R} \circ$
Fusion	$m \frac{S_{\bullet}}{R} \bullet$ $m \frac{S_o}{R} \circ$	$m(1-\Phi_{\uparrow}) \textcircled{2}$
β^+	$m \bullet$	$m(1-\varphi_{\beta^+}(\tau)) \frac{(1-q(R,S_o))}{q(R,S_o)} \frac{S_o}{S_{\bullet}} \circ$
β^-	$m \circ$	$m(1-\varphi_{\beta^-}(\tau)) \frac{q(R,S_o)}{(1-q(R,S_o))} \frac{S_{\bullet}}{S_o} \bullet$

■ **Table 3** Gluon W's Reactions (v3)

Reaction	Inputs	Outputs
Fission	$m \textcircled{2}$	$m(1-\Phi_{\downarrow}) \frac{S_{\bullet}}{R} \bullet$ $m(1-\Phi_{\downarrow}) \frac{S_o}{R} \circ$
Fusion	$m \frac{S_{\bullet}}{R(1-\Phi_{\uparrow})} \bullet$ $m \frac{S_o}{R(1-\Phi_{\uparrow})} \circ$	$m \textcircled{2}$
β^+	$m \bullet$	$m(1-\varphi_{\beta^+}(\tau)) \frac{(1-q(R,S_o))}{q(R,S_o)} \frac{S_o}{S_{\bullet}} \circ$
β^-	$m \circ$	$m(1-\varphi_{\beta^-}(\tau)) \frac{q(R,S_o)}{(1-q(R,S_o))} \frac{S_{\bullet}}{S_o} \bullet$

3 Stability Properties

Gluon W enjoys various stability properties, as proved in this section. These properties are stated in terms of the auxiliary concepts defined below.

■ Liabilities

$$L(R, S_{\circ}) = S_{\circ}P_{\circ}(R, S_{\circ}) = q(R, S_{\circ})R$$

■ Reserve Ratio:

$$r(R, S_{\circ}) = \frac{R}{L(R, S_{\circ})} = \frac{1}{q(R, S_{\circ})}$$

■ Equity:

$$E(R, S_{\circ}) = R - L(R, S_{\circ}) = (1 - q(R, S_{\circ}))R$$

Note the intuitive nature of these auxiliary concepts. The liabilities are the portion of the reserve backing neutrons, while the equity is the portion of the reserve backing protons, and the reserve ratio indicates by what factor the reserve is larger than the liabilities.

In the proofs, we often omit the arguments of functions for brevity. For instance, we write q instead of $q(R, S_{\circ})$.

The first stability property satisfied by Gluon W is that, in the normal reserve ratio range¹, users have no incentive to trade stablecoins outside the peg range in a secondary market. This is shown in Theorems 1 and 2.

► **Theorem 1 (Peg Maintenance - Upper Bound).** *Assume that the neutron target price $P_{\circ}^*$ remains constant between the consecutive actions of fission and β^+ ($\bullet \Rightarrow \circ$), where the number of neutrons obtained is N .*

If a user u wants to sell a neutron in the secondary market for a price P such that $P > (1 + \Phi(\tau))P_{\circ}^$, where $\Phi(\tau)$ is the effective fee for the operations of fission and then β^+ , then there is no rational user u^* who would buy from u .*

Furthermore, if $r(R, S_{\circ}) > \frac{1}{q^}$, $|\Delta(\tau)| \ll R + m$, $P_{\circ}N \ll R$, then the effective fee is $\Phi(\tau) \approx \frac{r}{(1-\Phi_{\downarrow})(1+(r-1)(1-\Phi_{\beta}^0))} - 1$.*

Proof. Assume, for the sake of contradiction, that such a rational u^* exists. Then u^* would have two options: to buy a neutron from u for P ; or to buy a neutron directly from the bank for P' where $P' = (1 + \Phi(\tau))P_{\circ}$. Note that the second option is available because the action of buying stable tokens from the bank always holds. By definition of $P_{\circ}$, $P' < (1 + \Phi(\tau))P_{\circ}^*$. Therefore, buying directly from the bank for P' would have been less costly than buying from u for P , and would have been the preferred option for a rational user. Hence u^* is irrational.

Now for the effective fee. We define purchasing N neutrons as first performing a fission and then a β^+ using the protons from the fission. Thus, we must derive an equation for the following question: how many m $\ominus$ do we need in order to purchase N neutrons? Let $P_{\circ}^{\downarrow}$ and $P_{\bullet}^{\downarrow}$ denote the neutron price and proton price after the fission, respectively.

The fission reaction of m $\ominus$ results in (see Table 3):

¹ In practice, there may be factors that may restrict interaction of the user with the reactor, such as blockchain congestion and high blockchain transaction fees. Such factors would have an effect on the ability of the reactor to maintain the peg range in secondary markets.

$$\frac{m(1-q)(1-\Phi_{\downarrow})}{P_{\bullet}} \bullet (\text{protons})$$

$$\frac{mq(1-\Phi_{\downarrow})}{P_{\circ}} \circ (\text{neutrons})$$

The β^+ reaction transforms (see Table 1):

$$\frac{\tilde{m}}{P_{\bullet}^{\downarrow}} \bullet \text{ into } \frac{\tilde{m}(1-\varphi_{\beta^+}(\tau))}{\tilde{P}_{\circ}^{\downarrow}} \circ$$

We can determine $\tilde{m}$ from the amount of protons resulting from the fission, assuming all of them will be used in β^+ :

$$\frac{\tilde{m}}{P_{\bullet}^{\downarrow}} = \frac{m(1-q)(1-\Phi_{\downarrow})}{P_{\bullet}}$$

$$\tilde{m} = \frac{m(1-q)(1-\Phi_{\downarrow})P_{\bullet}^{\downarrow}}{P_{\bullet}}$$

After both reactions, we will have in total N neutrons:

$$N = \frac{mq(1-\Phi_{\downarrow})}{P_{\circ}} + \frac{\tilde{m}(1-\varphi_{\beta^+}(\tau))}{P_{\circ}^{\downarrow}}$$

$$N = \frac{mq(1-\Phi_{\downarrow})}{P_{\circ}} + \frac{m(1-q)(1-\Phi_{\downarrow})P_{\bullet}^{\downarrow}(1-\varphi_{\beta^+}(\tau))}{P_{\bullet}P_{\circ}^{\downarrow}}$$

And now we can write m as a function of N :

$$m = \frac{P_{\bullet}P_{\circ}^{\downarrow}P_{\circ}N}{q(1-\Phi_{\downarrow})P_{\bullet}P_{\circ}^{\downarrow} + (1-q)(1-\Phi_{\downarrow})(1-\varphi_{\beta^+}(\tau))P_{\bullet}^{\downarrow}P_{\circ}}$$

Thus, by setting $N = 1$ we obtain the equation for calculating how many $\textcircled{2}$ are required for purchasing one neutron from the bank.

We can obtain the effective fee from the following:

$$m = (1 + \Phi(\tau))NP_{\circ}$$

$$\Phi(\tau) = \frac{P_{\bullet}P_{\circ}^{\downarrow}}{q(1-\Phi_{\downarrow})P_{\bullet}P_{\circ}^{\downarrow} + (1-q)(1-\Phi_{\downarrow})(1-\varphi_{\beta^+}(\tau))P_{\bullet}^{\downarrow}P_{\circ}} - 1$$

Since we assume that the neutron target price remains constant between operations and that $r > \frac{1}{q^*}$ so that the peg is maintained, we have that $P_{\circ}^{\downarrow} = P_{\circ} = P_{\circ}^*$. Since we also assume that $|\Delta(\tau)| \ll R + m$ then the beta-decay fee will be constant so that $\varphi_{\beta^+}(\tau) \approx \Phi_{\beta}^0$. Similarly, since we assume that $P_{\circ}N \ll R$ then $r^{\downarrow} = \alpha r \approx r$, where $\alpha = \frac{R+m}{R+(1-\Phi_{\downarrow})m}$. By applying these assumptions we obtain the following simplified effective fee:

$$\begin{aligned}
\Phi(\tau) &= \frac{\frac{(1-q)R}{S_\bullet}}{q(1-\Phi_\downarrow)(1-q)\frac{R}{S_\bullet} + (1-q)(1-\Phi_\downarrow)(1-\Phi_\beta^0)\frac{(1-q^\downarrow)R\alpha}{S_\bullet}} - 1 \\
&= \frac{1}{(1-\Phi_\downarrow)(q + \alpha(1-q^\downarrow)(1-\Phi_\beta^0))} - 1 \\
&= \frac{1}{(1-\Phi_\downarrow)(q + \alpha(1-\frac{q}{\alpha})(1-\Phi_\beta^0))} - 1 \\
&= \frac{1}{(1-\Phi_\downarrow)(q + q(\alpha r - 1)(1-\Phi_\beta^0))} - 1 \\
&\approx \frac{r}{(1-\Phi_\downarrow)(1+(r-1)(1-\Phi_\beta^0))} - 1
\end{aligned}$$

◀

► **Theorem 2** (Peg Maintenance - Lower Bound). *Assume that the neutron target price $P_\circ^*$ remains constant between the consecutive actions of β^- and fusion.*

If $r(R, S_\circ) > \frac{1}{q^}$, $|\Delta(\tau)| \ll R$, and a user u wants to buy a neutron in the secondary market for a price P such that $P < (1 - \Phi(\tau))P_\circ^*$, where $\Phi(\tau)$ is the effective fee for selling a neutron after the operations of partial β^- and fusion, then there is no rational user u^* who would sell to u .*

Furthermore, the effective fee will be $\Phi(\tau) = 1 - \frac{(1-\Phi_\uparrow)(1-\Phi_\beta^0)}{(1-q\Phi_\beta^0)}$.

Proof. Assume, for the sake of contradiction, that such a rational u^* exists. Then u^* would have had two options: to sell a neutron to u for P ; or to sell a neutron directly to the bank for P' where $P' = (1 - \Phi(\tau))P_\circ$. By definition of $P_\circ$ and the fact that $r(R, S_\circ) > \frac{1}{q^*}$, $P' = (1 - \Phi(\tau))P_\circ^*$. Therefore, selling directly to the bank for P' would have been more profitable than selling to u for P , and would have been the preferred option for a rational user. Hence u^* is irrational.

We define selling N neutrons in order to receive m $\mathfrak{Q}$ as first performing a partial β^- , to convert the necessary portion γN of our initial neutrons into protons, and then performing a fusion with these new protons and the remaining $(1 - \gamma)N$ neutrons. Thus, we must derive an equation for the following question: given N neutrons, how many m $\mathfrak{Q}$ do we get? Using Table 3, we have the following:

$$\begin{aligned}
\frac{\tilde{m}q^-}{P_\circ^-} \circ + \frac{\tilde{m}(1-q^-)}{P_\bullet^-} \bullet &= \tilde{m}(1-\Phi_\uparrow)\mathfrak{Q} = m\mathfrak{Q} \\
\frac{\gamma m_{\beta^-}}{P_\circ^-} \circ &= \frac{\gamma m_{\beta^-}(1-\varphi_{\beta^-}(\tau))}{P_\bullet^-} \bullet = \beta^-(\gamma N)\bullet
\end{aligned}$$

q^- , $P_\circ^-$, and $P_\bullet^-$ are the fusion ratio, neutron price, and proton price after the β^- reaction, respectively. To actually obtain the equation for m and γ we need to expand the terms in the equation according to their definition and use the simplified form. Accordingly, we now obtain the following results:

$$\begin{aligned}
(1 - \gamma)N &= \frac{\tilde{m}S_\circ^-}{R} \implies \tilde{m} = \frac{(1 - \gamma)NR}{S_\circ^-} \\
\implies m &= (1 - \Phi_\uparrow)\tilde{m} = \frac{(1 - \Phi_\uparrow)(1 - \gamma)NR}{S_\circ - \gamma N}
\end{aligned}$$

We can now obtain the effective fee by doing the following:

$$\begin{aligned}
m &= \frac{(1 - \Phi_{\uparrow})(1 - \gamma)NR}{(S_{\circ} - \gamma N)} \\
&= \frac{(1 - \Phi_{\uparrow})(1 - \gamma)NR}{(1 - \frac{\gamma N}{S_{\circ}})S_{\circ}} \frac{q}{q} \\
&= \frac{(1 - \Phi_{\uparrow})(1 - \gamma)NP_{\circ}}{q(1 - \frac{\gamma N}{S_{\circ}})} \\
&= (1 - \Phi(\tau))NP_{\circ} \\
\implies \Phi(\tau) &= 1 - \frac{(1 - \Phi_{\uparrow})(1 - \gamma)}{q(1 - \frac{\gamma N}{S_{\circ}})}
\end{aligned}$$

We must now derive equations for γ from the following:

$$\begin{aligned}
\beta^{-}(\gamma N) &= \frac{\gamma N(1 - \varphi_{\beta^{-}}(\tau))qS_{\bullet}}{(1 - q)S_{\circ}} \\
\beta^{-}(\gamma N) &= \frac{\tilde{m}S_{\bullet}^{-}}{R} = \frac{(1 - \gamma)NS_{\bullet}^{-}}{S_{\circ}^{-}} \\
S_{\bullet}^{-} &= S_{\bullet} + \beta^{-}(\gamma N) \\
S_{\circ}^{-} &= S_{\circ} - \gamma N
\end{aligned}$$

Since $|\Delta(\tau)| \ll R$ then $\varphi_{\beta^{-}}(\tau) \approx \Phi_{\beta}^0$

$$\begin{aligned}
\beta^{-}(\gamma N) &= \gamma N\beta^{-}(1) \\
\gamma N\beta^{-}(1) &= \frac{(1 - \gamma)N(S_{\bullet} + \gamma N\beta^{-}(1))}{(S_{\circ} - \gamma N)} \\
\gamma S_{\circ}\beta^{-}(1) - \gamma^2 N\beta^{-}(1) &= S_{\bullet} + \gamma N\beta^{-}(1) - \gamma S_{\bullet} - \gamma^2 N\beta^{-}(1) \\
\gamma S_{\circ}\beta^{-}(1) - \gamma N\beta^{-}(1) + \gamma S_{\bullet} &= S_{\bullet} \\
\gamma &= \frac{S_{\bullet}}{S_{\circ}\beta^{-}(1) - N\beta^{-}(1) + S_{\bullet}} \\
\gamma &= \frac{S_{\bullet}}{\beta^{-}(S_{\circ} - N) + S_{\bullet}}
\end{aligned}$$

We can then insert the derived equation for γ into the equation for $\Phi(\tau)$, where once we expand and simplify terms, we obtain the following equation for the effective-fee that is independent of the amount of neutrons:

$$\Phi(\tau) = 1 - \frac{(1 - \Phi_{\uparrow})(1 - \Phi_{\beta}^0)}{(1 - q\Phi_{\beta}^0)}$$

◀

The second stability property is that stablecoins remain pegged despite market crashes up to a magnitude that depends on the reserve ratio.

► **Theorem 3** (Peg Robustness during Market Crashes). *If $S_o > 0$, $r > \frac{1}{q^*}$ is the current reserve ratio, x and x' are the exchange rates before and after the crash, then the bank can tolerate a price crash of $\frac{r - \frac{1}{q^*}}{r}$ and the new stablecoin price would still be P_o^* .*

Proof. By the definition of P_o , the peg is maintained as long as $P_o^* \leq \frac{q^* R}{S_o}$. By the definition of P_o^* , reserve ratio and liabilities, this inequation can be simplified to $x' \leq q^* r x$. Since x and x' are the exchange rates of 1 unit of pegged currency in BCs, the exchange rates of 1 BC in pegged currency are $y = \frac{1}{x}$ and $y' = \frac{1}{x'}$. The inequation can then be rewritten as $y' \geq \frac{y}{q^* r}$. Therefore, $\frac{y - y'}{y} \leq \frac{r - \frac{1}{q^*}}{r}$. ◀

Another crucial property is that the reactor never becomes insolvent, which is a condition defined by having negative equity.

► **Theorem 4** (No Insolvency). *In all reactor states and for any exchange rate, $E(R, S_o) \geq 0$.*

Proof. Since $q \leq q^* < 1 \Rightarrow E(R, S_o) = (1 - q)R \geq 0$. ◀

The following theorem shows that, under the conditions of the theorem, the equity per proton increases after every operation due to collected fees. Note that this is one of the main incentives to hold protons.

► **Theorem 5** (Monotonically Increasing Equity per Reservecoin). *Assuming that the neutron target price, P_o^* , remains constant between actions, $R, S_o, S_\bullet > 0$, and that in the depeg state a user does not transmute neutrons to protons, for every action a , $\frac{E(R^a, S_o^a)}{S_\bullet^a} \geq \frac{E(R, S_o)}{S_\bullet}$, where $R^a, S_o^a, S_\bullet^a > 0$ are respectively, the reserve, number of stable tokens, and number of volatile tokens after action a .*

Proof. See Appendix A for the proof. ◀

4 Formal Verification

To obtain stronger formal guarantees and uncover potential issues in the informal proofs, we formalized the Gluon W protocol using the Rocq theorem prover. Rocq is an interactive theorem prover based on the Calculus of Inductive Constructions. We chose Rocq over other theorem provers such as Isabelle [3] or Lean [2] due to its ease of use, our familiarity with it, and the availability of useful libraries.

4.1 Design

As with any formal verification endeavor, it was critical to choose the right representation. Since the informal proofs involved substantial algebraic manipulation, we had to decide how to represent numbers in a way that aligned closely with the protocol definition while easing the formalization. We initially experimented with the Rocq standard library for floats [6] and rational numbers [7], but ultimately decided against this approach, as we would have to manually prove many basic algebraic facts and inequalities to make the libraries useful. Although using the float library might make sense when formalizing a protocol implementation in a language like Solidity, it proved inefficient for theorem proving. In the end, we chose to use the Rocq standard library for real numbers [8]. It offers the most expressive numerical representation, encompassing both rational and irrational numbers, and includes a rich set of lemmas that simplify algebraic manipulations and facilitate formalization.

Defined below are some of the relevant datatypes we used for the formalization. In the formalization, we make use of the word *Coin* instead of *Token*, or *Token*.

```

Definition StableCoins : Type := R.
Definition VolatileCoins : Type := R.
Definition BaseCoins : Type := R.
Definition ExchangeRate : Type := R.
Definition TimePeriod : Type := nat.
Definition Timestamp : Type := nat.
Definition QStar : Type := {x : R | 0 < x < 1}.
Definition FissionFee : Type := {x : R | 0 < x < 1}.
Definition FusionFee : Type := {x : R | 0 < x < 1}.
Definition BetaDecayFee0 : Type := {x : R | 0 < x}.
Definition BetaDecayFee1 : Type := {x : R | 0 < x}.

Record ReactorState :=
{
  baseCoins : BaseCoins;
  stableCoins : StableCoins;
  volatileCoins : VolatileCoins;
}.

Record StableCoinState :=
{
  reactorState : ReactorState;
  exchangeRate : ExchangeRate;
}.

Inductive Reaction : Type :=
| FissionReaction (baseCoins : BaseCoins)
| FusionReaction (baseCoins : BaseCoins)
| BetaDecayPosReaction (volatileCoins : VolatileCoins)
| BetaDecayNegReaction (stableCoins : StableCoins).

Definition Trace : Type := list (StableCoinState * Timestamp * Reaction).

Record State :=
{
  stableCoinState : StableCoinState;
  reactions : Trace;
}.

Inductive Action : Type :=
| BuyStableCoin
| SellStableCoin.

```

The protocol parameters q^* , $\Phi_\downarrow$, $\Phi_\uparrow$, Φ_β^0 , Φ_β^1 , and T are expressed as `QStar`, `FissionFee`, `FusionFee`, `BetaDecayFee0`, `BetaDecayFee1`, and `TimePeriod` respectively. Since most protocol parameters are bounded real numbers, we define them using Rocq's *Sigma Type*, which embeds the bounds as part of the type. This allows us to conveniently use the bounds in proofs via the `destruct` tactic. The record `ReactorState` keeps track of the number of `baseCoins`, `stableCoins`, and `volatileCoins` at any point in time and these fields mirror the variables R , $S_\circ$ and $S_\bullet$ defined above.

The record `StableCoinState` keeps track of the `reactorState`, and the `exchangeRate`. The field `exchangeRate` corresponds to $P_\circ^*$ defined above. The record `State` is composed of the `stableCoinState` and the `trace`. The `trace` is a history of all the reactions that occurred ordered from the most recent to least recent. Every entry in the list is a tuple of the `StableCoinState` before the reaction, the `Timestamp` when the reaction took place and the `Reaction` itself. The `trace` is useful for calculating $\Delta(\tau)$ in order to obtain the $\varphi_{\beta+}(\tau)$ and $\varphi_{\beta-}(\tau)$. We could have defined one record called the `State` that keeps track of everything but we adopted a composite structure to keep things organized and more importantly so the type `Trace` can refer to the `StableCoinState`.

The Gluon W protocol assumes that R , $S_\circ$, $S_\bullet$, and $P_\circ^*$ are always positive. These assumptions are formalized as **Axioms** and used throughout the proofs. While we initially encoded these facts in the types `BaseCoins`, `VolatileCoins`, `StableCoins`, and `ExchangeRate` using *Sigma Types*, this approach complicated function definitions by requiring explicit proofs for each return value. The Gluon W protocol definitions are formalized as functions analogous to their definitions above. In the formalization, we also define functions like `execute` that take in a `state : State`, `reaction : Reaction`, and `timestamp : Timestamp` as input and returns a new `State` as output as a result of executing the reaction. This function proves to be useful in the formalization of several theorems. We also define a function called `is_valid_state` that takes in a `state : State` as input and returns a **Prop** expressing the correctness of the record `State`.

4.2 Formalization of Theorems

The process of formalizing the theorems proved to be extremely fruitful. Apart from the obvious benefit of obtaining stronger guarantees, it helped us recognize issues in the informal proofs and the protocol itself and make adjustments accordingly. While working on the formal proofs, we made sure to replicate the informal proofs as closely as possible while striving for simplicity and clarity.

4.2.1 Theorem 1

The formal statement of the theorem is stated as follows:

```
Theorem peg_maintenance_upper_bound :
forall
  (timestamp : nat)
  (m : R)
  (s1 s2 : State)
  (offer : Offer)
  (betaDecayPosFeeVal : R)
  (gamma : R)
  (k rr fissFee b0 effFee primaryMarketOffer
   secondaryMarketOffer e0 e1 : R),
  (k > 0) → (m > 0) →
  (s2 = execute (s1) (FissionEvent (m) (timestamp)) →
   (e0 = target_price (s1.(stableCoinState)) →
    (e1 = target_price (s2.(stableCoinState)) →
     (betaDecayPosFeeVal = k * extract_value (betaDecayFee0)) →
     rr = reserve_ratio (s1.(stableCoinState)) →
     fissFee = extract_value (fissionFee) →
     b0 = extract_value (betaDecayFee0) →
     effFee = get_effective_fee (timestamp) (s1) (s2) (gamma)
      (betaDecayPosFeeVal) (BuyStableCoin) →
     offer.(action) = SellStableCoin →
     primaryMarketOffer = get_primary_market_offer (timestamp) (s1)
      (s2) (gamma) (betaDecayPosFeeVal) (BuyStableCoin) →
     secondaryMarketOffer = offer.(value) →
     (/extract_value (qStar)) < rr →
     ((1 + effFee) * target_price (s1.(stableCoinState))) <
      (offer.(value)) →
     (offer.(value)) →
     (effFee >= 0) →
     stablecoin_price (s1.(stableCoinState)) <> 0 →
     betaDecayPosFeeVal < 1 →
     e0 = e1 →
     get_rational_choice (BuyStableCoin) (primaryMarketOffer)
      (secondaryMarketOffer) = Primary
    ^
    effFee < ((rr) / ((1 - fissFee) * (1 + ((rr - 1) * (1 - (k * b0)))))) - 1.
```

The formal proof for this theorem involved two major goals. To prove the first goal that no rational user u^* would buy a stable token from the secondary market for a price higher than what he/she is getting in the primary market, we made use of a proof by contradiction mirroring the informal proof. We defined a function called `get_rational_choice` that returns the rational choice based on whether u^* is either buying or selling a stable token.

The second part of the informal proof derives an expression for m which gives us the number of base coins required to obtain N stable tokens. The proof then uses this expression to obtain an expression for $\Phi(\tau)$ and performs algebraic simplifications based on the assumptions.

The formal proof reflects a similar structure and we defined functions

`base_coins_for_n_stable_coins`, and `get_effective_fee` that reflect m , and $\Phi(\tau)$ respectively. We use the function `get_effective_fee` to obtain the $\Phi(\tau)$ for buying a stable token and perform algebraic manipulations to derive an upper-bound. Rather than deriving the value m from the protocol reactions, we make use of an elegant shortcut and prove the correctness of the function `base_coins_for_n_stable_coins`. We did so by defining another function called `stablecoins_from_m_basecoins` that returns the stable tokens obtained from performing *Fission* on the base coins and then *Beta Decay +* on the volatile tokens through the reactor equations. We proved that calling `stablecoins_from_m_basecoins` on the output of `base_coins_for_n_stable_coins` gives us the same stable tokens we required initially.

Formalizing this theorem revealed several subtle issues. First, we identified missing terms in the denominator when computing the value of m . In the original proof of Theorem 1, we incorrectly assumed that the price of the volatile token remains unchanged before and after the fission reaction—an assumption invalidated by Theorem 5, which shows that each reaction increases the equity per volatile token, i.e., its price. When deriving the value m in the informal proof we incorrectly assumed that the final expression is completely independent of m . This assumption is wrong as $\varphi_{\beta+}(\tau)$ depends on $\Delta(\tau)$ that takes into account the volume of the beta decay reaction that is yet to happen. This makes the $\varphi_{\beta+}(\tau)$ depend on m as m ultimately determines how many volatile tokens we get that would influence the value $\Delta\tau$. Obtaining an expression for m that is independent of m proved to be hard so we had to make assumptions about the value of the value of $\varphi_{\beta+}(\tau)$. In the informal proof we assume that $|\Delta(\tau)| \ll R + m$ which makes $\varphi_{\beta+}(\tau) \approx \Phi_{\beta}^0$. In the formalization we assume that $\varphi_{\beta+}(\tau) = k * \Phi_{\beta}^0$ where $k \in R$ and $k > 0$. The informal proof of Theorem 1 differs slightly from the formalized proof when it comes to the effective fee. In the informal proof, an approximation is derived for the effective fee whereas in the formalization an upper bound is derived as it's tricky to formalize the notion of approximately equal. By removing the term α since $\alpha \approx 1$, the informal proof does derive an upper bound but the actual value is very close to the upper bound hence the $\approx$ symbol is used.

4.2.2 Theorem 2

The formal statement of the theorem is stated as follows:

```
Theorem peg_maintenance_lower_bound :
forall
  (timestamp : nat)
  (s1 s2 : State)
  (offer : Offer)
  (primaryMarketOffer secondaryMarketOffer sC e0 e1 b0 rr k
   betaDecayNegFeeVal effFee gamma q fusFee s_0 p_s_0 : R),
(s2 = execute (s1) (BetaDecayNegEvent (sC)) (timestamp)) →
(q = fusion_ratio (s1.(stableCoinState))) →
(fusFee = extract_value (fusionFee)) →
```

```

(s_0 = get_stablecoins (s1.(stableCoinState).(reactorState))) →
(s_0 > 1) →
(p_s_0 = stablecoin_price (s1.(stableCoinState))) →
(e0 = target_price (s1.(stableCoinState))) →
(e1 = target_price (s2.(stableCoinState))) →
(e0 = e1) →
(b0 = extract_value (betaDecayFee0)) →
(rr = reserve_ratio (s1.(stableCoinState))) →
(rr > /extract_value (qStar)) →
(k > 0) →
(betaDecayNegFeeVal = k * b0) →
(betaDecayNegFeeVal < 1) →
(effFee = get_effective_fee (timestamp) (s1) (s2) (gamma)
(betaDecayNegFeeVal) (SellStableCoin)) →
(offer.(action) = BuyStableCoin) →
(primaryMarketOffer = get_primary_market_offer (timestamp) (s1)
(s2) (gamma)
(betaDecayNegFeeVal) (SellStableCoin)) →
(secondaryMarketOffer = offer.(value)) →
(((1 - effFee) * target_price (s1.(stableCoinState))) >
(secondaryMarketOffer)) →
(gamma = ((1 - q) * s_0) / (((s_0 - 1) * (1 - betaDecayNegFeeVal) * q) +
((1 - q) * s_0))) →
(get_rational_choice (SellStableCoin) (primaryMarketOffer)
(secondaryMarketOffer) = Primary)
^
(effFee = 1 - (((1 - fusFee) * (1 - betaDecayNegFeeVal))
/ (1 - (q * betaDecayNegFeeVal)))).

```

The formalization of Theorem 2 is similar to the formalization of Theorem 1 and the lessons learned from the formalization of Theorem 1 streamlined the process. The first part of the theorem statement that says no rational user u^* would sell a stable token in the secondary market for a price lower than what he/she is getting in the primary market is proved using a proof by contradiction. For deriving the value of the effective fee, we make use of the function `get_effective_fee` to obtain an expression for the effective fee and then perform algebraic manipulations based on the theorem assumptions to derive the final value. In this formal proof, we derive an exact value for the effective fee as opposed to an upper bound derived in the formal proof of Theorem 1.

4.2.3 Theorems 3 and 4

The formal statement of the theorems is stated as follows:

```

Theorem peg_robustness_during_market_crashes :
forall
  (s1 s2 : State),
  get_stablecoins (s1.(stableCoinState).(reactorState)) > 0 ∧
  reserve_ratio (s1.(stableCoinState)) > (1 / extract_value (qStar)) ∧
  target_price (s1.(stableCoinState)) > 0 ∧
  target_price (s2.(stableCoinState)) > 0 ∧
  (get_stablecoins (s1.(stableCoinState).(reactorState)) /
  get_basecoins (s1 (stableCoinState).(reactorState))) =
  (get_stablecoins (s2.(stableCoinState) (reactorState)) /
  get_basecoins (s2.(stableCoinState).(reactorState))) ∧
  base_coin_price_crash (s1) (s2) <= (reserve_ratio (s1.(stableCoinState))
  (1 / extract_value (qStar))) / (reserve_ratio (s1.(stableCoinState))) →
  stablecoin_price (s2.(stableCoinState)) =
  target_price (s2.(stableCoinState)).

Theorem insolvency :
forall (s : State),
equity (s.(stableCoinState)) >= 0.

```

The formal proofs of both these theorems were relatively straightforward and followed the same argument as the informal proofs.

4.2.4 Theorem 5

The formal statement of the theorem is stated as follows:

```
Theorem insolvency :
forall
  (s1 s2 : State)
  (e : Event)
  (t : Timestamp),
is_valid_state (s1) ∧
is_valid_state (s2) ∧
s2 = execute (s1) (e) (t) ∧
target_price (s1.(stableCoinState)) =
target_price (s2.(stableCoinState)) ∧
(is_depeg (s1.(stableCoinState)) → forall x, e <> BetaDecayNegEvent x) →
reservecoin_price (s2.(stableCoinState)) >=
reservecoin_price (s1.(stableCoinState)).
```

Formalizing this theorem took significant amount of time primarily because of the sheer number of cases to consider. In the first version of the informal proof, we were not considering the possibility that the value of the fusion ratio q might move from a pegged to a de-pegged state or vice-versa after performing the reaction. Working on the formal proof, uncovered this wrong assumption and we had to revise the proof to account for all possibilities which greatly increased the number of cases to consider. Since we had to account for all 4 reactions and each reaction having 4 sub-cases we had about 16 cases to consider. While working on one of these cases (the user performs a β^- reaction while the fusion ratio q is in a de-pegged state) we realized the theorem statement does not hold. To account for this, we had to modify the theorem statement.

4.3 General Remarks

Rocq's **Search** tactic proved to be quite useful when searching for relevant theorems and lemmas to apply. Most of the formal proofs are written in backward reasoning style where we apply theorems/lemmas whose conclusion is the goal we are trying to prove and then proceed to prove that the assumptions of the applied theorem/lemma also hold. We made extensive use of tactics like **lra**, **nra**, and **lia** to resolve easy goals. We also defined a **Module** called **HelperLemmas** where we proved various algebraic inequalities/equalities related to real numbers. These helper lemmas proved to be quite useful throughout the formalization.

5 Related Work

The conception of Gluon W took into account lessons learned from Djed [10,11]. The main improvements in comparison to Djed are:

- In Djed users are forbidden to redeem reservecoins when the reserve ratio is below a threshold. Although this is beneficial to stablecoin holders, it is very undesirable for potential reservecoin buyers, since they may remain unable to redeem their reservecoins for an unbounded amount of time. In Gluon W, users are never forbidden to perform any of the four reactions. In particular, users can always redeem their protons by first transmuting part of them to neutrons and then fusing their protons and neutrons back to base coins.

- In Djed, the minting and burning fees are fixed. Therefore, Djed can only tolerate oracle delays or manipulations up to a certain limit. To have a sufficiently high tolerance, the fees need to be high. But high fees are unattractive to users. In Gluon W, on the other hand, the beta decay fees depend on the volume of recent transactions and on the volume of the current transaction. This allows the fee to be small for small transactions in times of low volume, but increase in times of high volume to protect against large transactions that would attempt to exploit oracle delays or manipulations.

However, to achieve these improvements, some stability properties that were provable for Djed are not provable for Gluon W. They are:

- No Bank Runs for Stablecoins: this property is not provable for Gluon W because, due to the variable fee, neutron holders who redeem their neutrons earlier would enjoy a lower fee. Therefore, it is rational to run to be early. Nevertheless, like Djed, apart from the fees, Gluon W still ensures that all neutron holders are treated equally.
- No Reserve Drainage: although in Gluon W it is still impossible to outrightly drain the reserve, it is possible, under certain conditions, to shift part of the portion of reserves backing neutrons (or protons) to the portion backing protons (or neutrons, respectively). This means that Gluon W may behave counterintuitively with respect to how much of reserves are backing neutrons and how much of reserves are backing protons when the reserve ratio is low.
- Bounded Dilution: this property required Djed to forbid minting of reservecoins when the reserve ratio was above a threshold. In Gluon W, a deliberate choice was made, for the sake of improved user experience, not to prohibit any action.

6 Implementations

The Gluon W protocol has been implemented in Ergoscript and deployed on the Ergo blockchain. It can be used in <https://gluon.gold>. This is the first ever decentralized gold-pegged stablecoin: the price of neutrons in this deployment is pegged to the price of 1g of gold.

The source code of the smart contracts can be inspected in <https://github.com/StabilityNexus/Gluon-Ergo-Contracts>. The source code of the web UI is available at <https://github.com/StabilityNexus/Gluon-Ergo-UI>. A typescript SDK to interact with the smart contracts can be used as described in <https://github.com/StabilityNexus/Gluon-Ergo-SDK>. A simulator of the protocol, written in Python, is available in <https://github.com/StabilityNexus/Gluon-Simulation>. And the formalization of the stability theorems and their proofs can be found in <https://github.com/StabilityNexus/Gluon-Formalization-Coq>.

7 Conclusions and Future Work

The Gluon W protocol is the first of a family of protocols whose main characteristic is the ability to split (*fission*) a volatile asset into a stable sub-asset and an even more volatile sub-asset and merge (*fusion*) them back. It is only the second stablecoin protocol, after Djed, to have stability theorems proven and formally verified. Besides its theoretical foundations, Gluon W is currently being battle-tested in practice, as the basis for the first decentralized gold-pegged stablecoin. In our current and future work, we are exploring variations of this protocol, particularly by looking at different options for Equation 1 and alternative approaches for beta

decays. The formalization could be improved as well. For the formal proof of Theorem 2, we assume that the expression for γ equal the one described in the informal Proof 2 rather than deriving this value from the equations of the reactions. The next step would be to define a function called `gamma` that returns the expression γ and prove its correctness similar to how we proved the correctness of the function `base_coins_for_n_stable_coins` in the formalization of Theorem 1. There are opportunities for reducing the redundancies in the proof scripts as well. While working on the formal proofs, we noticed patterns in the type of algebraic manipulations performed. The next step would be to identify these patterns and if they occur repeatedly, abstract them away either as helper lemmas or new tactics using `Ltac`.

References

- 1 The Maker Protocol: MakerDAO's Multi-Collateral Dai (MCD) System. [Online], 2017. <https://makerdao.com/en/whitepaper/>.
- 2 Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, page 625–635, Berlin, Heidelberg, 2021. Springer-Verlag. doi:10.1007/978-3-030-79876-5_37.
- 3 Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL – A Proof Assistant for Higher-Order Logic*, volume 2283 of *LNCS*. Springer, 2002.
- 4 Rocq. Rocq Theorem Prover. <https://rocq-prover.org/>, Retrieved in July/2025.
- 5 Robert Sams. A Note on Cryptocurrency Stabilisation: Seigniorage Shares. [Online], 2014. <https://blog.bitmex.com/wp-content/uploads/2018/06/A-Note-on-Cryptocurrency-Stabilisation-Seigniorage-Shares.pdf>.
- 6 The Rocq Development Team. *Rocq Standard Library: Module Stdlib.Floats.Floats*. Inria Rocquencourt, 2025. Provides machine-level floating-point types and operations. URL: <https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.Floats.Floats.html>.
- 7 The Rocq Development Team. *Rocq Standard Library: Module Stdlib.QArith.QArith*. Inria Rocquencourt, 2025. Provides rational number arithmetic on the type $\mathbb{Q}$ (fractions of integers). URL: <https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.QArith.QArith.html>.
- 8 The Rocq Development Team. *Rocq Standard Library: Module Stdlib.Reals.Reals*. Inria Rocquencourt, 2025. Provides the real number axiomatization and core analysis theorems on the type $\mathbb{R}$. URL: <https://rocq-prover.org/doc/V9.0.0/stdlib/Stdlib.Reals.Reals.html>.
- 9 Bruno Woltzenlogel Paleo. *Stablecoin*, pages 1–5. Springer Berlin Heidelberg, Berlin, Heidelberg, 2019. doi:10.1007/978-3-642-27739-9_1671-1.
- 10 Joachim Zahmentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged algorithmic stablecoin. Cryptology ePrint Archive, Report 2021/1069, 2021. <https://eprint.iacr.org/2021/1069>.
- 11 Joachim Zahmentferner, Dmytro Kaidalov, Jean-Frédéric Etienne, and Javier Díaz. Djed: A formally verified crypto-backed pegged autonomous stablecoin. In *IEEE International Conference on Blockchain and Cryptocurrency*, 2023. <https://icbc2023.ieee-icbc.org/program/icbc-2023-final-program>.

A

 Proof of Theorem 5

► **Theorem 5** (Monotonically Increasing Equity per Reservecoin). *Assuming that the neutron target price, P_o^* , remains constant between actions, $R, S_o, S_\bullet > 0$, and that in the depeg state a user does not transmute neutrons to protons, for every action a , $\frac{E(R^a, S_o^a)}{S_\bullet^a} \geq \frac{E(R, S_o)}{S_\bullet}$, where $R^a, S_o^a, S_\bullet^a > 0$ are respectively, the reserve, number of stable tokens, and number of volatile tokens after action a .*

Proof. Let $A = \{F^\downarrow, F^\uparrow, \beta^+, \beta^-\}$ be the set of four fundamental actions in the Gluon protocol: fission, fusion, beta-decay plus, and beta-decay minus, respectively. Since we assume that the neutron target price remains constant between actions, for each possible value of the fusion ratio, q , before and after the action, we show what the equity per volatile token is when applying the corresponding action on an arbitrary reactor state $|R, S_o, S_\bullet\rangle \in \mathbb{R}_{>0}^3$.

■ $a = F^\downarrow$

$$\text{— } F^\downarrow |R, S_o, S_\bullet\rangle = |R + F_\bullet^\downarrow, S_o + F_o^\downarrow, S_\bullet + F_\bullet^\downarrow\rangle$$

—

$$P_\bullet^\downarrow = \frac{E^\downarrow}{S_\bullet^\downarrow} \tag{1}$$

$$= \frac{(1 - q^\downarrow)R^\downarrow}{S_\bullet^\downarrow} \tag{2}$$

$$= \frac{(1 - q^\downarrow)(R + F_\bullet^\downarrow)}{S_\bullet + F_\bullet^\downarrow} \tag{3}$$

$$= \frac{(1 - q^\downarrow)(R + m)}{S_\bullet + \frac{(1 - \Phi_\downarrow)mS_\bullet}{R}} \tag{4}$$

$$= (1 - q^\downarrow) \frac{R}{S_\bullet} \frac{(1 + \frac{m}{R})}{(1 + \frac{(1 - \Phi_\downarrow)m}{R})} \tag{5}$$

$$= \frac{(1 - q^\downarrow)R\alpha}{S_\bullet}, \alpha > 1 \tag{6}$$

— Case 1: $q = q^*, q^\downarrow = q^*$

$$= \frac{(1 - q^*)R\alpha}{S_\bullet} \tag{7}$$

$$> \frac{E}{S_\bullet} \tag{8}$$

$$= P_\bullet \tag{9}$$

Since $q^* < \frac{P_o^* S_o}{R\alpha} < \frac{P_o^* S_o}{R}$, this case is possible if $P_o^* > \frac{q^* R}{S_o}$.

— Case 2: $q = q^*, q^\downarrow = \frac{P_o^* S_o^\downarrow}{R^\downarrow \alpha} = \frac{P_o^* S_o}{R\alpha} < q^*$

$$= \frac{(1 - \frac{P_o^* S_o}{R\alpha})R\alpha}{S_\bullet} \tag{10}$$

$$> \frac{(1 - q^*)R\alpha}{S_\bullet} \tag{11}$$

$$> \frac{E}{S_\bullet} \tag{12}$$

$$= P_\bullet \tag{13}$$

Since $\frac{P_o^* S_o}{R\alpha} < q^* < \frac{P_o^* S_o}{R}$, this case is possible if $\alpha > \frac{P_o^* S_o}{q^* R} > 1$.

$$\blacksquare \text{ Case 3: } q = \frac{P_o^* S_o}{R}, q^\downarrow = \frac{P_o^* S_o^\downarrow}{R^\downarrow} = \frac{P_o^* S_o}{R\alpha} < q$$

$$= \frac{(1 - \frac{P_o^* S_o}{R\alpha})R\alpha}{S_\bullet} \quad (14)$$

$$= \frac{(1 - \frac{q}{\alpha})R\alpha}{S_\bullet} \quad (15)$$

$$> \frac{(1 - q)R\alpha}{S_\bullet} \quad (16)$$

$$> \frac{E}{S_\bullet} \quad (17)$$

$$= P_\bullet \quad (18)$$

This case is possible by definition since $\alpha > 1$.

$$\blacksquare \text{ Case 4: } q = \frac{P_o^* S_o}{R}, q^\downarrow = q^* < \frac{P_o^* S_o^\uparrow}{R^\uparrow} = \frac{P_o^* S_o}{R\alpha} < q$$

Since we assume the neutron target price remains constant between actions, this case is not possible.

$$\blacksquare a = F^\uparrow$$

$$\blacksquare F^\uparrow |R, S_o, S_\bullet\rangle = |R - F_\bullet^\uparrow, S_o - F_o^\uparrow, S_\bullet - F_\bullet^\uparrow\rangle$$

■

$$P_\bullet^\uparrow = \frac{E^\uparrow}{S_\bullet^\uparrow} \quad (19)$$

$$= \frac{(1 - q^\uparrow)R^\uparrow}{S_\bullet^\uparrow} \quad (20)$$

$$= \frac{(1 - q^\uparrow)(R - F_\bullet^\uparrow)}{S_\bullet - F_\bullet^\uparrow} \quad (21)$$

$$= \frac{(1 - q^\uparrow)(R - (1 - \Phi_\uparrow)m)}{S_\bullet - \frac{mS_\bullet}{R}} \quad (22)$$

$$= (1 - q^\uparrow) \frac{R}{S_\bullet} \frac{(1 - \frac{(1 - \Phi_\uparrow)m}{R})}{(1 - \frac{m}{R})} \quad (23)$$

$$= \frac{(1 - q^\uparrow)R\alpha}{S_\bullet}, \alpha > 1 \quad (24)$$

$$\blacksquare \text{ Case 1: } q = q^*, q^\uparrow = q^*$$

$$= \frac{(1 - q^*)R\alpha}{S_\bullet} \quad (25)$$

$$> \frac{E}{S_\bullet} \quad (26)$$

$$= P_\bullet \quad (27)$$

Since $q^* < \frac{P_o^* S_o}{R\alpha} < \frac{P_o^* S_o}{R}$, this case is possible if $P_o^* > \frac{q^* R}{S_o}$.

- Case 2: $q = q^*, q^\uparrow = \frac{P_o^* S_o^\uparrow}{R^\uparrow} = \frac{P_o^* S_o}{R\alpha} < q^*$

$$= \frac{(1 - \frac{P_o^* S_o}{R\alpha})R\alpha}{S_\bullet} \quad (28)$$

$$> \frac{(1 - q^*)R\alpha}{S_\bullet} \quad (29)$$

$$> \frac{E}{S_\bullet} \quad (30)$$

$$= P_\bullet \quad (31)$$

Since $\frac{P_o^* S_o}{R\alpha} < q^* < \frac{P_o^* S_o}{R}$, this case is possible if $\alpha > \frac{P_o^* S_o}{q^* R} > 1$.

- Case 3: $q = \frac{P_o^* S_o}{R}, q^\uparrow = \frac{P_o^* S_o^\uparrow}{R^\uparrow} = \frac{P_o^* S_o}{R\alpha} < q$

$$= \frac{(1 - \frac{P_o^* S_o}{R\alpha})R\alpha}{S_\bullet} \quad (32)$$

$$= \frac{(1 - \frac{q}{\alpha})R\alpha}{S_\bullet} \quad (33)$$

$$> \frac{(1 - q)R\alpha}{S_\bullet} \quad (34)$$

$$> \frac{E}{S_\bullet} \quad (35)$$

$$= P_\bullet \quad (36)$$

This case is possible by definition since $\alpha > 1$.

- Case 4: $q = \frac{P_o^* S_o}{R}, q^\uparrow = q^* < \frac{P_o^* S_o^\uparrow}{R^\uparrow} = \frac{P_o^* S_o}{R\alpha} < q$

Since we assume the neutron target price remains constant between actions, this case is not possible.

- $a = \beta^+$

- $\beta^+ |R, S_o, S_\bullet\rangle = |R, S_o + \beta_o^+, S_\bullet - \beta_\bullet^+\rangle$

■

$$P_\bullet^+ = \frac{E^+}{S_\bullet^+} \quad (37)$$

$$= \frac{(1 - q^+)R^+}{S_\bullet^+} \quad (38)$$

$$= \frac{(1 - q^+)R}{S_\bullet - \beta_\bullet^+} \quad (39)$$

$$= \frac{(1 - q^+)R}{S_\bullet - m} \quad (40)$$

$$= (1 - q^+) \frac{R}{S_\bullet \left(1 - \frac{m}{S_\bullet}\right)} \quad (41)$$

$$= \frac{(1 - q^+)R\alpha}{S_\bullet}, \alpha > 1 \quad (42)$$

- Case 1: $q = q^*, q^+ = q^*$

$$= \frac{(1 - q^*)R\alpha}{S_\bullet} \quad (43)$$

$$> \frac{E}{S_\bullet} \quad (44)$$

$$= P_\bullet \quad (45)$$

Since $q^* < \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R} < \frac{P_\bullet^* S_\bullet}{R}$, this case is possible if $P_\bullet^* > \frac{q^* R}{S_\bullet}$, where $\tilde{\alpha} = (1 + \frac{m(1 - \varphi_{\beta^+}(\tau))(1 - q)}{q S_\bullet}) > 1$.

- Case 2: $q = q^*, q^+ = \frac{P_\bullet^* S_\bullet^+}{R^+} = \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R} < q^*$

Since we assume the neutron target price remains constant between actions, this case is not possible.

- Case 3: $q = \frac{P_\bullet^* S_\bullet}{R}, q^+ = \frac{P_\bullet^* S_\bullet^+}{R^+} = \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R} < q^*$

$$= \frac{(1 - \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R})R\alpha}{S_\bullet} \quad (46)$$

$$= \frac{(1 - q\tilde{\alpha})R}{S_\bullet} \quad (47)$$

$$= \frac{(1 - q(1 + \frac{m(1 - \varphi_{\beta^+}(\tau))(1 - q)}{q S_\bullet}))R\alpha}{S_\bullet} \quad (48)$$

$$= \frac{((1 - q) - \frac{m(1 - \varphi_{\beta^+}(\tau))(1 - q)}{S_\bullet})R\alpha}{S_\bullet} \quad (49)$$

$$= \frac{(1 - q)R}{S_\bullet} \frac{(1 - \frac{m(1 - \varphi_{\beta^+}(\tau))}{S_\bullet})}{(1 - \frac{m}{S_\bullet})} \quad (50)$$

$$> \frac{E}{S_\bullet} \quad (51)$$

$$= P_\bullet \quad (52)$$

Since $\frac{P_\bullet^* S_\bullet}{R} < \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R} < q^*$, this case is possible if $1 < \tilde{\alpha} < \frac{q^* R}{P_\bullet^* S_\bullet}$.

- Case 4: $q = \frac{P_\bullet^* S_\bullet}{R}, q^+ = q^* < \frac{P_\bullet^* S_\bullet^+}{R^+} = \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R}$

$$= \frac{(1 - q^*)R\alpha}{S_\bullet} \quad (53)$$

$$> (1 - \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R}) \frac{R\alpha}{S_\bullet} \quad (54)$$

$$= (1 - q(1 + \frac{m(1 - \varphi_{\beta^+}(\tau))(1 - q)}{q S_\bullet})) \frac{R\alpha}{S_\bullet} \quad (55)$$

$$= (1 - q) \frac{R}{S_\bullet} \frac{(1 - \frac{m(1 - \varphi_{\beta^+}(\tau))}{S_\bullet})}{(1 - \frac{m}{S_\bullet})} \quad (56)$$

$$> \frac{E}{S_\bullet} \quad (57)$$

$$= P_\bullet \quad (58)$$

Since $\frac{P_\bullet^* S_\bullet}{R} < \frac{P_\bullet^* S_\bullet \tilde{\alpha}}{R} < q^*$, this case is possible if $1 < \frac{q^* R}{P_\bullet^* S_\bullet} < \tilde{\alpha}$.

- $a = \beta^-$

$$\beta^- |R, S_o, S_\bullet\rangle = |R, S_o - \beta_o^-, S_\bullet + \beta_\bullet^-\rangle$$

■

$$P_\bullet^- = \frac{E^-}{S_\bullet^-} \quad (59)$$

$$= \frac{(1 - q^-)R^-}{S_\bullet^-} \quad (60)$$

$$= \frac{(1 - q^-)R}{S_\bullet + \beta_\bullet^-} \quad (61)$$

$$= \frac{(1 - q^-)R}{S_\bullet + \frac{m(1 - \varphi_{\beta^-}(\tau))qS_\bullet}{(1 - q)S_o}} \quad (62)$$

$$= \frac{(1 - q^-)R}{S_\bullet \left(1 + \frac{m(1 - \varphi_{\beta^-}(\tau))q}{(1 - q)S_o}\right)} \quad (63)$$

$$= \frac{(1 - q^-)R\alpha}{S_\bullet}, \alpha < 1 \quad (64)$$

- Case 1: $q = q^*, q^- = q^*$
By assumption, we do not consider this case since it would result in a decrease in the equity per volatile token.
- Case 2: $q = q^*, q^- = \frac{P_o^* S_o^-}{R^-} = \frac{P_o^* S_o \tilde{\alpha}}{R} < q^*$
By assumption, we do not consider this case since it would result in a decrease in the equity per volatile token, where $\tilde{\alpha} = (1 - \frac{m}{S_o}) < 1$.
- Case 3: $q = \frac{P_o^* S_o}{R}, q^- = \frac{P_o^* S_o^-}{R^-} = \frac{P_o^* S_o \tilde{\alpha}}{R} < q$

$$= \frac{(1 - \frac{P_o^* S_o \tilde{\alpha}}{R})R\alpha}{S_\bullet} \quad (65)$$

$$= \frac{(1 - q\tilde{\alpha})R\alpha}{S_\bullet} \quad (66)$$

$$= \frac{(1 - q(1 - \frac{m}{S_o}))R}{(1 + \frac{m(1 - \varphi_{\beta^-}(\tau))q}{(1 - q)S_o})S_\bullet} \quad (67)$$

$$= \frac{(1 - q)R}{S_\bullet} \frac{(1 - q(1 - \frac{m}{S_o}))}{(1 - q(1 - \frac{m(1 - \varphi_{\beta^-}(\tau))}{S_o}))} \quad (68)$$

$$> \frac{E}{S_\bullet} \quad (69)$$

$$= P_\bullet \quad (70)$$

This case is possible by definition since $\tilde{\alpha} < 1$.

- Case 4: $q = \frac{P_o^* S_o}{R}, q^- = q^* < \frac{P_o^* S_o^-}{R^-} = \frac{P_o^* S_o \tilde{\alpha}}{R}$
Since we assume the neutron target price remains constant between actions, this case is not possible.

◀